Figure 7.10 An inefficient implementation of an LFU cache. When all elements of a list with `n` entries have the same value for counter, and one element at the end of the list is accessed, we will have to perform `n-1` node swaps to bubble up the updated entry to the front of the list.

We could implement our LFU from the code of in section 7.4, just changing the eviction policy, and therefore the order of elements in the list. When we add a new entry, we set its counter to 1, and we could insert it at the tail of the list, while on a cache hit we could increment the node's counter and move it towards the head of the list, till we find another node with a larger counter.

That would work correctly, indeed. But *how efficient* would it be? Would there be a way to improve the performance of our cache?

